

Лекция 1 Введение в тестирование ПО

Жизненный цикл разработки ПО (SDLC - Software Development Lifecycle)

Жизненный цикл программного обеспечения (software lifecycle): Период времени, начинающийся с момента появления концепции программного обеспечения и заканчивающийся тогда, когда дальнейшее использование программного обеспечения невозможно. Жизненный цикл программного обеспечения обычно включает в себя следующие этапы: концепт, описание требований, дизайн, реализация, тестирование, инсталляция и наладка, эксплуатация и поддержка и, иногда, этап вывода из эксплуатации. Данные фазы могут накладываться друг на друга или проводиться итерационно. (ISTQB)

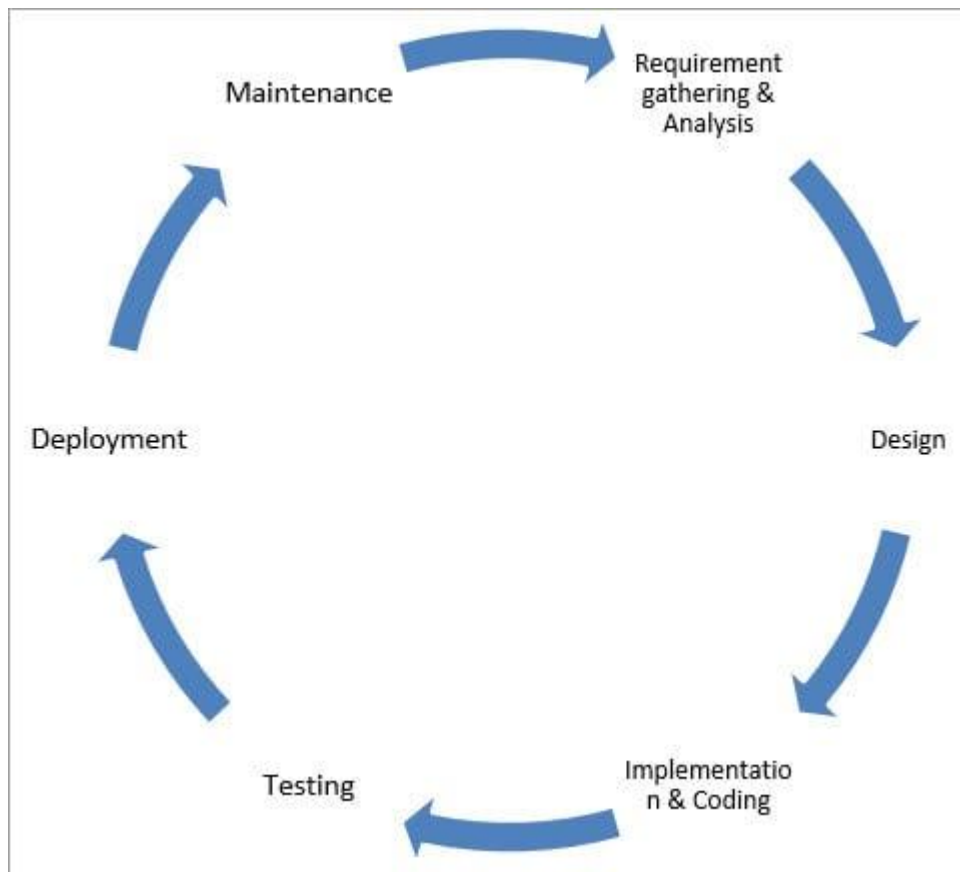
ISTQB (International Software Testing Qualifications Board) — некоммерческая организация, которая занимается определением принципов развития сферы тестирования ПО.

Период времени от концепции до первоначальной версии известен как жизненный цикл разработки, который является частью жизненного цикла программного обеспечения. С момента первого запуска система переходит в эксплуатацию (функционирование). Система остается в эксплуатации до момента прекращения использования. (ГОСТ 56920)

SDLC - это систематизированный процесс, этапы которого охватывают полный жизненный цикл программного обеспечения (Software Lifecycle) и который определяет различные этапы разработки программного обеспечения для создания высококачественного программного обеспечения, отвечающего ожиданиям клиентов и для улучшения эффективности разработки. Разработка системы должна быть завершена в заранее определенные сроки и стоимость. Каждая фаза жизненного цикла SDLC имеет свой собственный процесс и результаты, которые используются в следующей фазе.

Обычно он делится на шесть-восемь шагов, но менеджеры проектов могут объединять, декомпозировать или пропускать шаги, в зависимости от скоупа проекта.

В разных источниках фазы немного отличаются, но глобально суть везде одинакова.



<https://www.softwaretestinghelp.com/wp-content/qa/uploads/2018/04/SDLC-Cycle.jpg>

Фазы SDLC:

- **Сбор и анализ требований** (Requirement Gathering and Analysis): На этом этапе от клиентов собирается вся необходимая информация для разработки продукта в соответствии с их ожиданиями. Любые неясности должны быть разрешены сразу на этом этапе. Бизнес-аналитик и менеджер проекта назначили встречу с заказчиком, чтобы собрать всю информацию, например, что заказчик хочет построить, кто будет конечным пользователем, какова цель продукта. Перед созданием продукта очень важно понимание или знание продукта. Например, клиент хочет иметь приложение, которое включает денежные транзакции. В этом случае требование должно быть четким, например, какие транзакции будут выполняться, как они будут проводиться, в какой валюте они будут проводиться и т. д. После того,

как сбор требований завершен, проводится анализ для проверки возможности разработки продукта. После четкого понимания требования создается документ SRS (Спецификация требований к программному обеспечению). Этот документ должен быть полностью понятен разработчикам, а также должен быть рассмотрен заказчиком для использования в будущем;

- **Дизайн** (Design): На этом этапе требования, собранные в документе SRS, используются в качестве входных данных, и создается архитектура программного обеспечения, которая используется для реализации разработки системы.
- **Разработка** (Implementation or Coding): Реализация / кодирование начинается, как только разработчик получает Design document. Дизайн программного обеспечения переведен в исходный код. На этом этапе реализуются все компоненты программного обеспечения;
- **Тестирование** (Testing): В классическом понимании, как фаза SDLC, тестирование начинается после завершения кодирования и выпуска модулей для тестирования. Однако, на самом деле тестирование присутствует практически на всех этапах SDLC, начиная с анализа собранных требований. На этом этапе разработанное программное обеспечение тщательно тестируется, и все обнаруженные дефекты передаются разработчикам для их исправления. Повторное тестирование, регрессионное тестирование проводится до тех пор, пока программное обеспечение не будет соответствовать ожиданиям клиента. Тестировщики обращаются к документу SRS, чтобы убедиться, что программное обеспечение соответствует стандарту заказчика;
- **Развертывание** (Deployment): После тестирования продукта он развертывается в производственной среде или выполняется первое UAT (пользовательское приемочное тестирование), в зависимости от ожиданий клиента. В случае UAT создается копия производственной

среды, и заказчик вместе с разработчиками выполняет тестирование.

Если клиент остается доволен, то предоставляет согласие на [релиз](#);

- **Поддержка (Maintenance):** Основное внимание на этом этапе SDLC уделяется обеспечению того, чтобы потребности продолжали удовлетворяться и чтобы система продолжала работать в соответствии со спецификацией, упомянутой в первом этапе. После того, как система развернута и клиенты начинают использовать разработанную систему, следует 3 вида активностей:
 - Исправление ошибок;
 - Обновление;
 - Улучшение.

Источник:

https://vladislavremeev.gitbook.io/qa_bible/sdlc-i-stlc/zhiznennyi-cikl-razrabotki-po-sdlc-software-development-lifecycle

Заметка от автора курса

В некоторых источниках дополнительно выделяют следующие этапы:

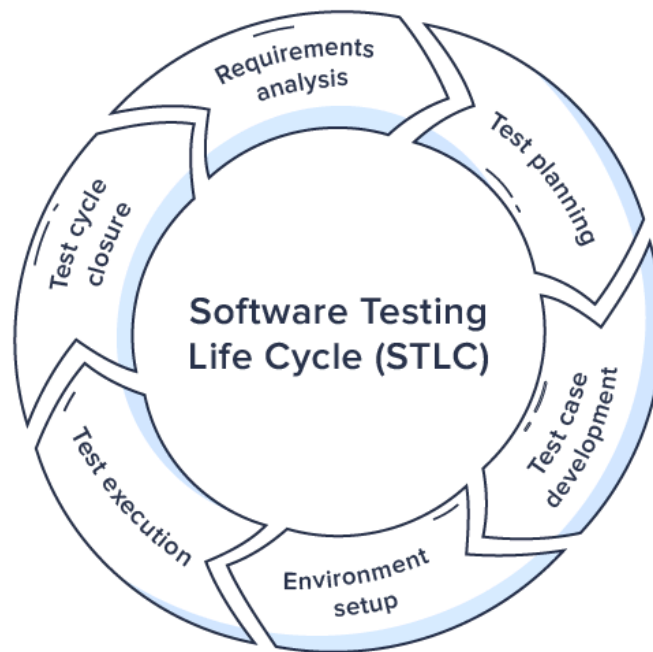
- **Идея, или планирование системы**, который предшествует этапу "Анализ требований". Представляет собой определение проблемы, которую поможет решить разрабатываемый продукт.
- **Ввод в эксплуатацию** - синоним этапа "Развертывание".
- **Вывод из эксплуатации**, который завершает SDLC. Например, при устаревании продукта и прекращении его поддержки, необходимо сформировать процесс вывода продукта из эксплуатации, который включает в себя: сохранение информации, удаление данных, удаление аппаратного и программного обеспечения

Жизненный цикл тестирования ПО (STLC - Software Testing Lifecycle)

STLC - это процесс тестирования, который включает в себя определенную последовательность шагов, чтобы гарантировать достижение целей в области качества. В процессе STLC каждое действие выполняется планомерно и

систематически. Каждый этап имеет разные цели и результаты. У разных организаций разные этапы STLC, однако основа остается прежней.

Фазы STLC



bamboo agile

<https://bambooagile.eu/insights/wp-content/uploads/2021/01/In-article-STLC-copy.png>

STLC имеет несколько взаимосвязанных фаз и в целом очень похож на SDLC. Эти фазы являются последовательными и называются:

- **Анализ требований** (Requirement Analysis): один из важнейших этапов, потому что именно на нем можно почти бесплатно исправить недостатки проекта. Этап анализа требований также определяет потенциальную потребность в автоматизированном тестировании и позволяет производить экономические расчеты затрат на рабочую силу на основе оценки проекта. На этом же этапе обсуждаются и документируются критерии начала и окончания тестирования
- **Планирование тестирования** (Test Planning): на этом этапе формируется план тестирования, т.е. мы определяем действия и ресурсы, которые

помогут достичь целей тестирования (участники и их роли, инструменты, окружение). Во время планирования мы также пытаемся определить метрики, метод сбора и отслеживания этих метрик. План составляют исходя из требований, тестовой стратегии и анализа рисков

- **Разработка тест-кейсов** (Test Case Development): подразумевает использование ручного и автоматизированного тестирования для достижения полного охвата функциональности программного обеспечения, при этом процесс основан на заранее установленных требованиях. Чаще всего тест-кейсы для автоматического тестирования пишутся отдельно, так как кейсы для ручного тестирования описаны в виде шпаргалок (cheat sheets)
- **Настройка тестовой среды** (Test Environment Setup): в плане тестирования четко указано, какую тестовую среду следует использовать. На этом этапе STLC настраиваются операционные системы и виртуальные машины, развертываются инструменты тестирования, такие как Selenium, Katalon Studio, а также тестовая среда и базы данных проекта. Мы также обращаемся с запросами к DevOps и администраторам, если требуется поддержка
- **Выполнение тестов** (Test Execution): тесты выполняются на основе готовой тестовой документации и правильно настроенной тестовой среды. Все результаты тестирования регистрируются в Системе управления тестированием. Отрицательно пройденные тесты, в которых фактический результат отличается от ожидаемого, регистрируются как ошибки и передаются команде разработчиков на доработку с последующей пере проверкой после исправления
- **Завершение цикла испытаний** (Test Cycle Closure): окончательная генерация отчетов о тестировании для клиента. Они должны включать затраченное время, процент обнаруженных ошибок и положительных результатов тестирования, общее количество обнаруженных и исправленных ошибок. Что касается отдела тестирования, то это

момент для анализа его работы, подведения итогов, анализа его продуктивности и возможности внести предложения по улучшению качества тестирования

Разница STLC и SDLC

STLC и SDLC тесно связаны друг с другом, но они одновременно преследуют разные задачи с одной и той же целью, а именно:

- сбор требований в желаемой форме и разработка заявленной функциональности (SDLC);
- анализ требований, помощь клиенту и команде разработчиков и подтверждение качества реализованной функциональности (STLC).

Общая цель - удовлетворение клиента и получение максимально возможного балла на этапах верификации и валидации.

Источник:

https://vladislavermeev.gitbook.io/qa_bible/sdlc-i-stlc/zhiznennyi-cikl-testirovaniya-po-stlc-software-testing-lifecycle

Принципы тестирования

1. Тестирование демонстрирует наличие дефектов (Testing shows presence of defects)
2. Искрывающее тестирование недостижимо (Exhaustive testing is not possible)
3. Раннее тестирование (Early testing)
4. Скопление/кластеризация дефектов (Defect clustering)
5. Парадокс пестицида/тесты устаревают (Pesticide paradox/Tests wear out)
6. Тестирование зависит от контекста (Testing is context dependent)
7. Заблуждение об отсутствии ошибок (Absence of errors fallacy)

Принцип 1. Тестирование показывает наличие дефектов

Тестирование может показать, что дефекты присутствуют, но не может доказать, что дефектов больше нет.

Сколько бы успешных тестов вы не провели, вы не можете утверждать, что нет таких тестов, которые не нашли бы ошибку.

Принцип 2. Исчерпывающее тестирование невозможно

Для проведения исчерпывающего тестирования придется протестировать все возможные входные значения и все пути выполнения программы, в большинстве случаев число таких вариаций стремится к бесконечности или просто на порядки превосходит отведенное время и бюджет. Даже при условии использования полного перебора при тест-дизайне, мы искусственно ограничиваем область тестирования, поэтому его все еще не получится назвать исчерпывающим. Вместо попыток «протестировать все» нам нужен некий подход к тестированию (стратегия), который обеспечит правильный объем тестирования для данного проекта, данных заказчиков (и других заинтересованных лиц) и данного продукта.

При определении, какой объем тестирования достаточен, **необходимо учитывать уровень риска, включая технические риски и риски, связанные с бизнесом, и такие ограничения проекта как время и бюджет**. Оценка и управление рисками - одна из наиболее важных активностей в любом проекте.

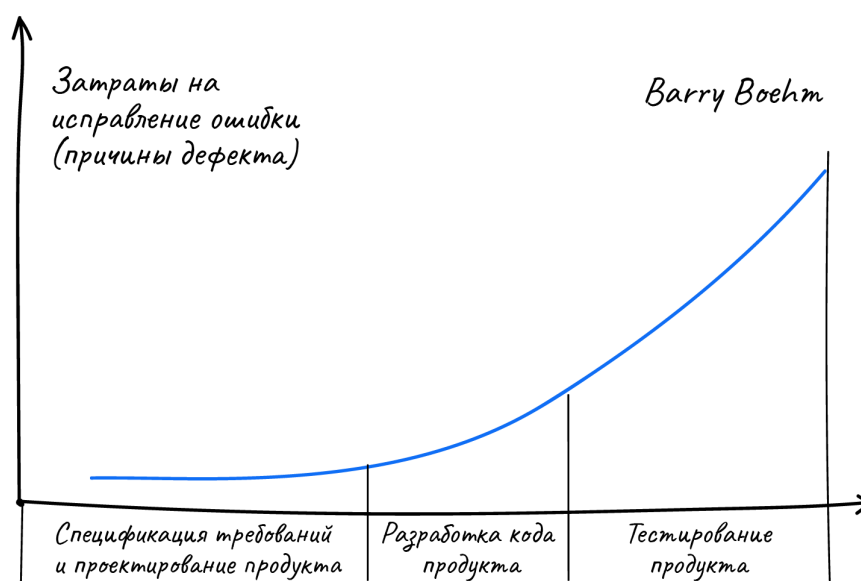
Принцип 3. Раннее тестирование

Тестовые активности должны начинаться как можно раньше в SDLC, а именно когда сформированы требования.

Этот принцип связан с понятием «цена дефекта» (cost of defect). Цена дефекта существенно растет на протяжении жизненного цикла разработки ПО. Чем раньше обнаружен дефект, тем быстрее, проще и дешевле его исправить. Дефект, найденный в требованиях, обходится дешевле всего.



Кривая Бозма: экспоненциальный рост стоимости исправления дефектов



источник картинки:

<https://cdn2.hexlet.io/store/derivatives/original/e9958bb0ee31528359e0b2ce291b4519.png>

Еще одно важное преимущество раннего тестирования - экономия времени. Тестовые активности могут начинаться еще до того, как написана первая строка кода. По мере того, как готовятся требования и спецификации, тестировщики могут приступать к разработке и ревью тест-кейсов. И когда появится первая тестовая версия, можно будет сразу приступить к выполнению тестов.

Принцип 4. Скопление/кластеризация дефектов

Небольшое количество модулей содержит большинство дефектов, обнаруженных на этапе предрелизного тестирования, или же демонстрируют наибольшее количество отказов на этапе эксплуатации.

Многие тестировщики наблюдали такой эффект - дефекты «кучкуются». Это может происходить потому, что определенная область кода особенно сложна и запутана, или потому, что внесение изменений производит «эффект домино». Это знание часто используется для оценки рисков при планировании тестов - тестировщики фокусируются на известных «проблемных зонах». Также полезно проводить анализ первопричин (root cause analysis), чтобы предотвратить повторное появление дефектов, обнаружить причины возникновения скоплений дефектов и спрогнозировать потенциальные скопления дефектов в будущем.

Принцип 5. Парадокс пестицида/тесты устаревают

Boris Beizer в своей книге Software Testing Techniques объяснил парадокс пестицида как феномен, согласно которому чем больше вы тестируете ПО, тем более невосприимчивым оно становится к имеющимся тестам, т.е.

- каждый метод и набор тестов, который используется для предотвращения или поиска ошибок, может оставлять часть не найденных ошибок, против которых эти методы и тесты неэффективны;
- имеющиеся тесты устаревают после исправления дефекта и не могут обнаружить новые.

Из чего следует, что набор тестов, тестовых данных и подходов нужно постоянно пересматривать и улучшать для выявления не найденных ошибок, а также необходимо обновлять тесты и тестовые данные после исправления уже найденных дефектов.

Принцип 6. Тестирование зависит от контекста

Тестирование выполняется по-разному, в зависимости от контекста. Например, тестирование систем, критических с точки зрения безопасности, проводится иначе, чем тестирование сайта интернет-магазина.

Этот принцип тесно связан с понятием риска. Что такое риск? Риск - это потенциальная проблема. У риска есть вероятность (likelihood) - она всегда выше 0 и ниже 100% - и есть влияние (impact) - те негативные последствия, которых мы опасаемся. Анализируя риски, мы всегда взвешиваем эти два аспекта: вероятность и влияние.

То же можно сказать и о мире ПО: разные системы связаны с различными уровнями риска, влияние того или иного дефекта также сильно варьируется. Одни проблемы довольно тривиальны, другие могут дорого обойтись и привести к большим

потерям денег, времени, деловой репутации, а в некоторых случаях даже привести к травмам и смерти.

Уровень риска влияет на выбор методологий, техник и типов тестирования.

Принцип 7. Заблуждение об отсутствии ошибок

Нахождение и исправление дефектов бесполезно, если построенная система неудобна для использования и не соответствует нуждам и ожиданиям пользователей. Заказчики ПО - люди и организации, которые покупают и используют его, чтобы выполнять свои повседневные задачи - на самом деле совершенно не интересуются дефектами и их количеством, кроме тех случаев, когда они непосредственно сталкиваются с нестабильностью продукта. Им также неинтересно, насколько ПО соответствует формальным требованиям, которые были задокументированы. Пользователи ПО более заинтересованы в том, чтобы оно помогало им эффективно выполнять задачи. ПО должно отвечать их потребностям, и именно с этой точки зрения они его оценивают.

Даже если вы выполнили все тесты и ошибок не обнаружили, это еще не гарантия того, что ПО будет соответствовать нуждам и ожиданиям пользователей.

Иначе говоря, верификация не равна валидации.

Источник: https://vladislavermeev.gitbook.io/qa_bible/obshee/principy-testirovaniya

QA/QC/Testing

Обеспечение Качества (quality assurance) - это совокупность запланированных и систематических процессов и действий поддержки, необходимых для обеспечения надлежащего уровня уверенности в том, что процесс или рабочий продукт удовлетворяет установленным техническим требованиям или требованиям к качеству. Достигается это сочетанием методов, стандартов, инструментов и навыков, признанных как соответствующая практика. Процесс Обеспечения Качества использует результаты тестирования и другую информацию для анализа, оценки и информирования о любой проблеме (включая любой риск) в проектировании, планировании или выполнении процессов программной инженерии. (ГОСТ 56920)

Контроль качества (quality control): Рабочие методы и активности, нацеленные на выполнение требований к качеству, являющиеся частью управления качеством. (ISO 8402)

Тестирование (testing): Процесс, содержащий в себе все активности жизненного цикла, как динамические, так и статические, касающиеся планирования, подготовки и оценки программного продукта и связанных с этим результатов работ с целью определить, что они соответствуют описанным требованиям, показать, что они подходят для заявленных целей и для определения дефектов. (ISTQB)

Тестирование (testing): Набор операций, проводимых для обеспечения выявления и/или оценки свойств одного или более элементов тестирования.

Примечание - Действия тестирования могут включать в себя планирование, подготовку, выполнение, создание отчетов и менеджмент, поскольку все они направлены на тестирование. (ГОСТ 56920)

Обеспечение качества (QA - Quality Assurance) - это часть Quality Management - совокупность мероприятий, охватывающих все технологические этапы разработки, выпуска и поддержки ПО, предпринимаемых на разных стадиях жизненного цикла ПО, для обеспечения требуемого уровня качества выпускаемого продукта. QA обеспечивает создание правильных процессов для получения в результате качественного продукта.

Это также означает создание процессов **контроля качества (QC - Quality Control)**, которые в свою очередь гарантируют, что процессы, установленные QA, соблюдаются. То есть QC - это часть QA - процесс установления стандартов и проверки, что ПО сделано правильно. Цель контроля качества - проверить, соблюдалась ли предписанная модель. Это может быть достигнуто путем проведения аудитов и определения того, следовала ли команда определенной модели для достижения качества.

Активности QA проходят на всем протяжении SDLC: на этапе построения, анализа и улучшения процессов, формирования релизных политик, риск менеджмента и прочих how-to's.

QC подключаются на этапе составления критериев качества, [quality gate](#)-ов, метрик и способов оценки.

Тестирование - это деятельность, направленная на предоставление всем заинтересованным лицам исчерпывающих сведений о текущем качестве продукта и любых остаточных рисках, а также на сведение к минимуму дефектов, которые может обнаружить конечный пользователь, при заданных сроках и бюджете. (с) словами автора статьи

Тестирование программного обеспечения направлено на предоставление информации о программном продукте и нахождении максимально возможного числа дефектов на ранних этапах процесса разработки при заданных ограничениях стоимости и графика разработки.

Основными целями **тестирования** как части QC являются:

- предоставление информации о качестве элемента тестирования и любых остаточных рисках относительно того, до какой степени элемент тестирования был проверен;
- обнаружение дефектов в элементе тестирования до его передачи в эксплуатацию;
- смягчение рисков получения продукта низкого качества заинтересованными сторонами.

Вышеупомянутая информация может использоваться в нескольких целях, включая:

- улучшение элемента тестирования путем устранения дефектов;
- улучшение управленческих решений, предоставляя как основание для решений информацию о качестве и рисках;
- улучшение процессов в организации, особо выделяя процессы, которые позволяют дефектам возникать и/или оставаться скрытыми там, где они могут быть обнаружены.

Давайте разберемся **на примере создания мобильного приложения**, потому что определения не всегда отражают суть:

1. В рамках **тестирования** мы выполним проверки и задокументируем дефекты, убедимся, что продукт соответствует требованиям.
2. В рамках **контроля качества** мы проанализируем полученные данные и убедимся, что соблюдены все требования, предъявляемые к качеству как продукта, так и самого процесса. Мы должны убедиться, что уровень качества нашего продукта высокий и он готов к релизу.
3. В рамках **обеспечения качества** мы формируем процесс QA для соответствия стандартам качества на всех этапах SDLC, еще до этапа

создания нашего продукта, который будет минимизировать количество дефектов и предупреждать их.

Основные отличия процессов:

1. Контроль качества и тестирование (как его часть) направлены на продукт, а обеспечение качества на процесс.
2. Тестирование и контроль качества являются контролирующими мерами, а обеспечение качества - превентивными, или предупреждающими.

Как понять, что тестировщик хорошо сделал свою работу?

Бытует мнение, что основная задача тестировщиков - сломать продукт, на самом деле тот уже приходит на тестирование с дефектами, одна из задач тестировщика как раз их выявить.

Понять, что тестировщик выполнил свою работу хорошо можно по факту выполнения следующих задач:

- продукт проверен на соответствие требованиям;
- сведено к минимуму количество дефектов, которые обнаружит конечный пользователь;
- предоставлена отчетность по актуальному качеству продукта и любых остаточных рисках заинтересованным лицам.

Верификация и валидация

Верификация - доказанное объективными результатами исследования подтверждение того, что определенные требования были выполнены. [ISTQB Glossary]

Валидация - подтверждение путем экспертизы того, что рабочий продукт соответствует потребностям заинтересованной стороны. [ISTQB Glossary]

Верификация показывает то, что продукт разработан согласно требованиям к нему и соответствует им (нуждам заказчика). Отвечает на вопрос: правильно ли мы разрабатываем продукт?

Чаще всего под верификацией подразумевают статические методы проверки (без запуска кода), которые включают в себя анализ требований, дизайна, кода.

Валидация показывает то, что продукт разработан согласно ожиданиям конечного пользователя (нуждам пользователя). Отвечает на вопрос: правильный ли продукт мы разработали?

Валидация включает в себя динамические проверки (с запуском кода) и тестирование уже рабочего продукта.



Вопросы по лекции

1. Что такое SDLC?
2. Что такое STLC?
3. Большую часть критических дефектов находят в ограниченном количестве модулей. 80% проблем содержатся в 20% модулей. Это принцип тестирования ...?
4. Верно ли следующее утверждение? *Тестирование может показать, что дефекты присутствуют, но не может доказать, что их нет.*
5. Расположите фазы в порядке увеличения стоимости исправления бага:
 - Разработка и тестирование
 - Ввод в эксплуатацию
 - Сбор и анализ требований
6. Верификация чаще всего представляет собой ...?
7. Верно ли следующее утверждение? *Тестирование ПО – проверка соответствия между реальным и ожидаемым поведением программы.*

8. Какие процессы концентрируются больше на продукте, чем на процессе?
- Тестирование
 - Менеджмент качества (QM)
 - Контроль качества (QC)
 - Обеспечение качества (QA)
9. Какой из процессов является процессно-ориентированным?
- Нет правильного ответа
 - Контроль качества
 - Тестирование
 - Обеспечение качества
10. Верно ли следующее утверждение? *В рамках **контроля качества** мы формируем процесс QA для соответствия стандартам качества на всех этапах SDLC, еще до этапа создания нашего продукта, который будет минимизировать количество дефектов и предупреждать их.*